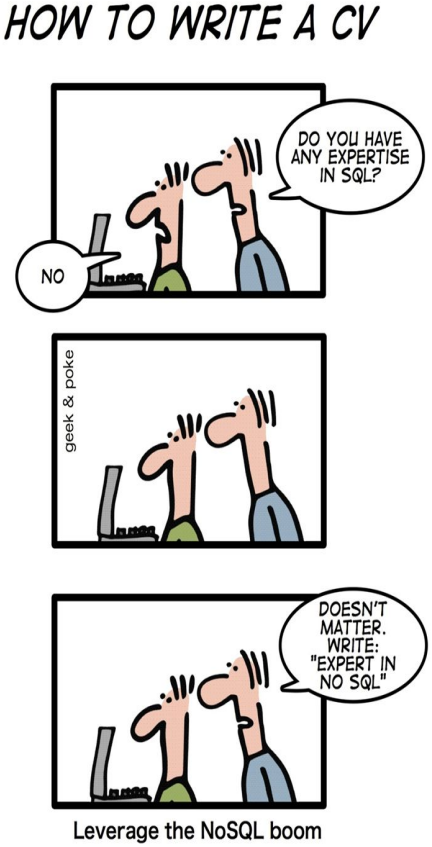
NoSQL!

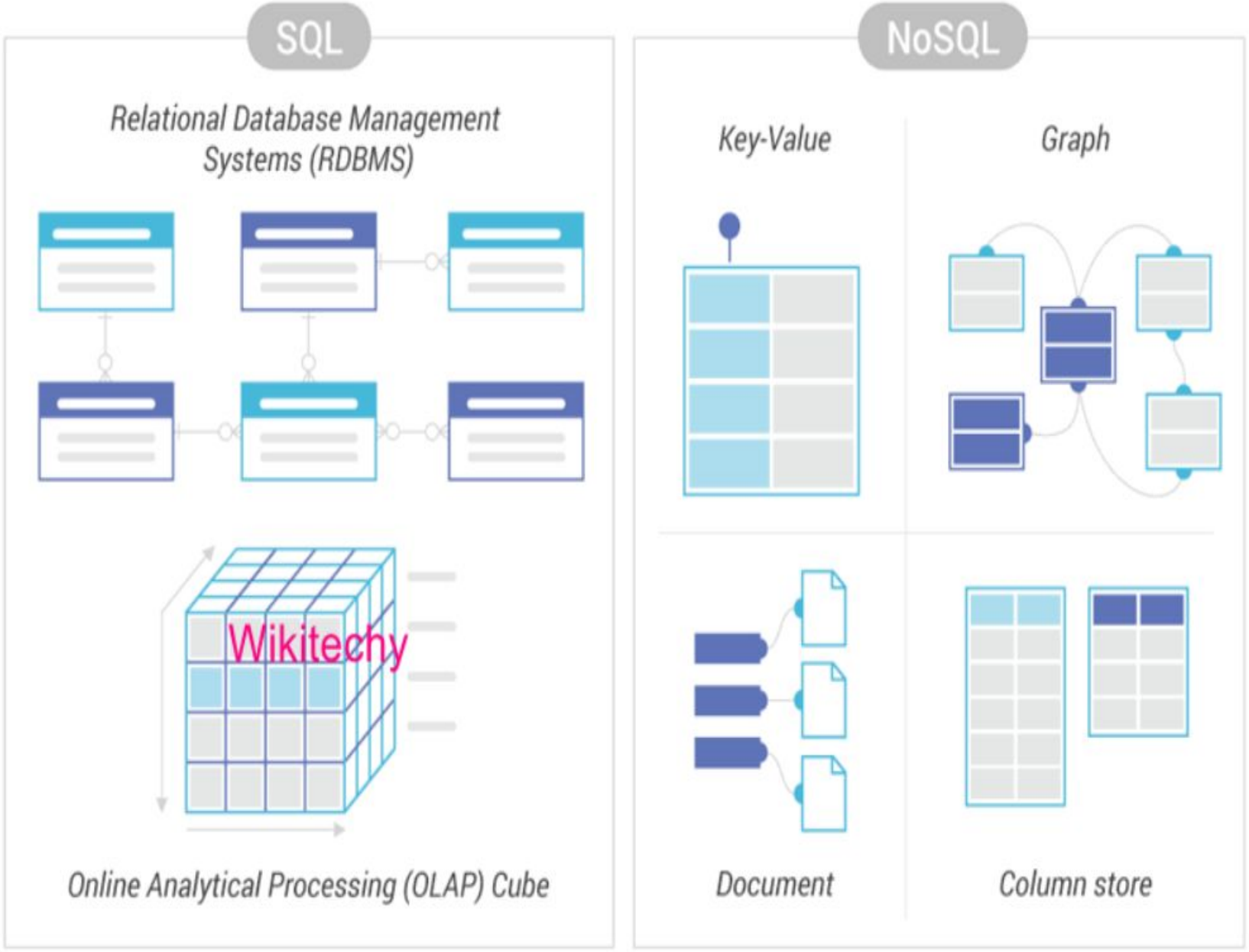
What is NoSQL Database ?

- A NoSQL database (Not Only SQL) is a type of database management system that provides a mechanism for storing, retrieving, and managing data that does not follow the traditional relational database model.
- Unlike relational databases, NoSQL databases are designed to handle unstructured, semi-structured, or structured data, providing greater flexibility and scalability for certain types of applications.

mongoDB Firebase CouchDB



SQL Vs NoSQL Database



RDBMS Characteristics

- Data stored in columns and tables
- Relationships represented by data
- Data Manipulation Language
- Data Definition Language
- Transactions
- Abstraction from physical layer
- Applications specify what, not how
- Physical layer can change without modifying applications
  - Create indexes to support queries
  - In Memory databases

No SQL?

- NoSQL stands for:
  - No Relational
  - No RDBMS
  - Not Only SQL
- NoSQL is an umbrella term for all databases and data stores that don't follow the RDBMS principles
  - A class of products
  - A collection of several (related) concepts about data storage and manipulation
  - Often related to large data sets

Where does NoSQL come from?

- Non-relational DBMSs are not new
- But NoSQL represents a new incarnation
  - Due to massively scalable Internet applications
  - Based on distributed and parallel computing
- Development
  - Starts with Google
  - First research paper published in 2003
  - Continues also thanks to Lucene's developers/Apache (Hadoop) and Amazon (Dynamo)
  - Then a lot of products and interests came from Facebook, Netfix, Yahoo, eBay, Hulu, IBM, and many more

Characteristics of NoSQL Database

1) **Schema-less Design**

   NoSQL databases typically do not enforce a fixed schema. This allows developers to insert data without first defining its structure, making them more adaptable to evolving data models.

2) **Variety of Data Models**

   NoSQL databases support various data models, including document-oriented, key-value pairs, wide-column stores, and graph databases. This versatility allows developers to choose the best-fit data model for their specific application needs.

Characteristics of NoSQL Database

3) Horizontal Scalability

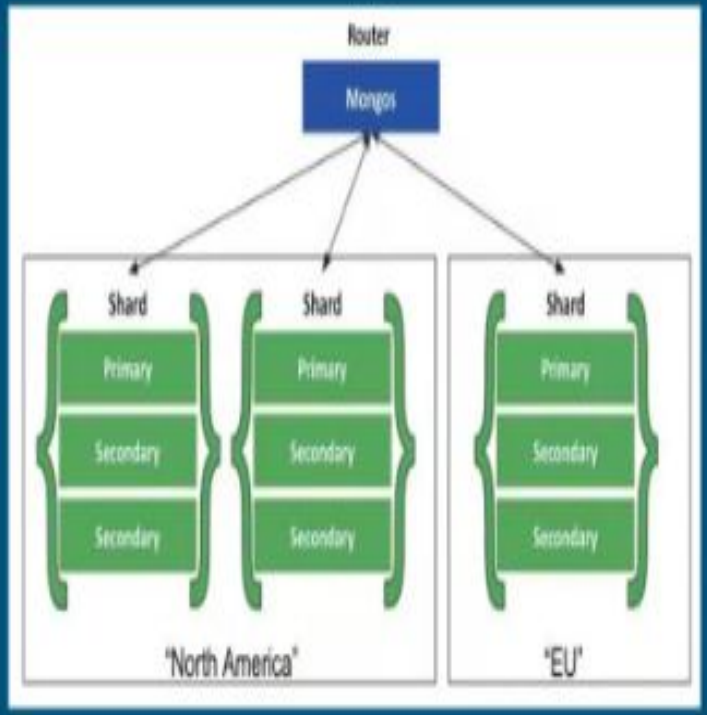
NoSQL databases are designed to scale horizontally, meaning they can handle increased data volumes and traffic by adding more servers or nodes to the database system. This is in contrast to the vertical scaling often associated with traditional relational databases.



Characteristics of NoSQL Database

4) Distributed Architecture

Many NoSQL databases are built to operate in distributed environments, allowing them to distribute data across multiple nodes or servers. This design enhances fault tolerance and ensures data availability.



Characteristics of NoSQL Database

5) **Dynamic and Unstructured Data**

NoSQL databases are well-suited for managing unstructured or semi-structured data, such as JSON or XML documents. This makes them versatile in handling diverse data formats.



Types of NoSQL

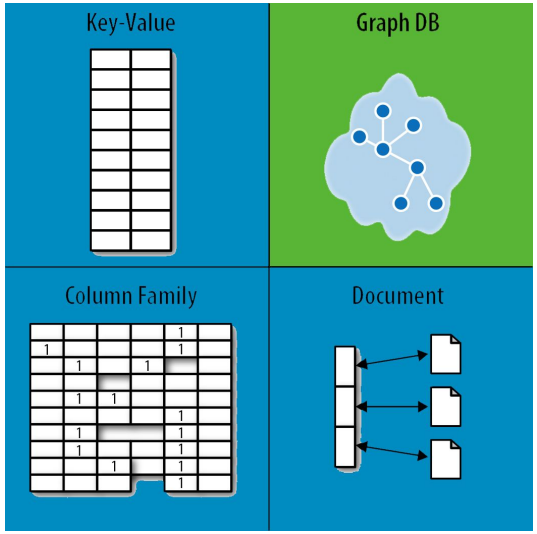
Categories of NoSQL Database

NoSQL databases can be broadly categorized into four main types, each with its own unique data model and characteristics. These categories are based on the way data is organized and stored within the database. The four main categories of NoSQL databases are:

1) Document-oriented database
2) Key-Value stores
3) Column-oriented database
4) Graph-based database

NoSQL Database Types

Discussing NoSQL databases is complicated because there are a variety of types:

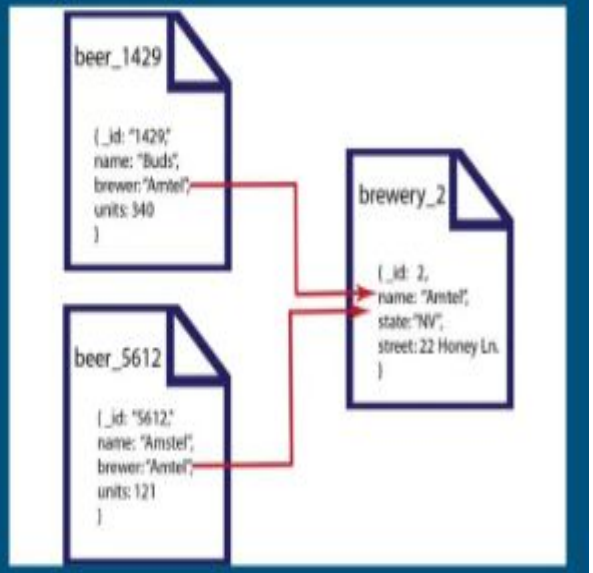


- Sorted ordered Column Store
  - Optimized for queries over large datasets, and store columns of data together, instead of rows
- Document databases:
  - pair each key with a complex data structure known as a document.
- Key-Value Store :
  - are the simplest NoSQL databases. Every single item in the database is stored as an attribute name (or 'key'), together with its value.
- Graph Databases :
  - are used to store information about networks of data, such as social connections.

Document-Oriented

Document-oriented Database

- Data is stored as documents, typically in JSON or BSON format.
- Each document is a self-contained unit that may contain nested structures.
- Documents are often organized in collections.
- Examples : MongoDB, CouchDB, Elasticsearch



Document Databases (Document Store)

- Documents
  - Loosely structured sets of key/value pairs in documents, e.g., XML, JSON, BSON
  - Encapsulate and encode data in some standard formats or encodings
  - Are addressed in the database via a unique key
  - Documents are treated as a whole, avoiding splitting a document into its constituent name/value pairs
- Allow documents retrieving by keys or contents
- Notable for:
  - MongoDB (used in FourSquare, Github, and more)
  - CouchDB (used in Apple, BBC, Canonical, Cern, and more)

Document Databases (Document Store)

- The central concept is the notion of a "document“ which corresponds to a row in RDBMS.
- A document comes in some standard formats like JSON (BSON).
- Documents are addressed in the database via a unique *key* that represents that document.
- The database offers an API or query language that retrieves documents based on their contents.
- Documents are schema free, i.e., different documents can have structures and schema that differ from one another. (An RDBMS requires that each row contain the same columns.)

What is a Document DB?

- Document databases store documents in the value part of the key-value store where:
  - Documents are indexed using a BTree
  - and queried using a JavaScript query engine

```
{
  name: "sue",                          ← field: value
  age: 26,                              ← field: value
  status: "A",                          ← field: value
  groups: [ "news", "sports" ]          ← field: value
}
```

Document Databases

What is a Document DB?

```
{
  "name": "Phil",
  "age": 26,
  "status": "A"
}
```

```
{
  "name": "Phil",
  "age": 26,
  "status": "A",
  "citiesVisited" : ["Chicago", "LA", "San Francisco"]
}
```

- Documents have differences in their attributes
- But belongs to the same collection
- This is different from relational databases where columns:
  - Stores the same type of values
  - Or null

Document Databases, JSON

```
{
    _id: ObjectId("51156a1e056d6f966f268f81"),
    type: "Article",
    author: "Derick Rethans",
    title: "Introduction to Document Databases with MongoDB",
    date: ISODate("2013-04-24T16:26:31.911Z"),
    body: "This arti…"
},
{
    _id: ObjectId("51156a1e056d6f966f268f82"),
    type: "Book",
    author: "Derick Rethans",
    title: "php|architect's Guide to Date and Time Programming with PHP",
    isbn: "978-0-9738621-5-7"
}
```

MongoDB

How to Use MongoDB

Install: Check the MongoDB website
https://docs.mongodb.com/manual/installation/

Create collection and insert a document

```
db.users.insert({name: “Jack”, email:
“jack@example.com”});
```

Retrieve all/some documents

```
db.users.find();
db.users.find({name: “Jack”});
```

Update

```
db.users.update({name: "Jack"}, {$set: {hobby:
"cooking"}});
updateOne, updateMany, replaceOne
```

Delete

```
db.users.remove({name: "Alex"});
deleteOne, deleteMany
```

MongoDB

Schema Validation

- You can still explicitly create collections and enforce schema validation

```
db.createCollection("students", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: [ "name", "year", "major", "address" ],
    properties: {
      name: {
        bsonType: "string",
        description: "must be a string and is required" },
    …
    }
  }}
}
```

Indexing

- Like RDBMS, document databases use indexes to speed up some queries



- MongoDB uses B-tree as an index structure

Key-Value

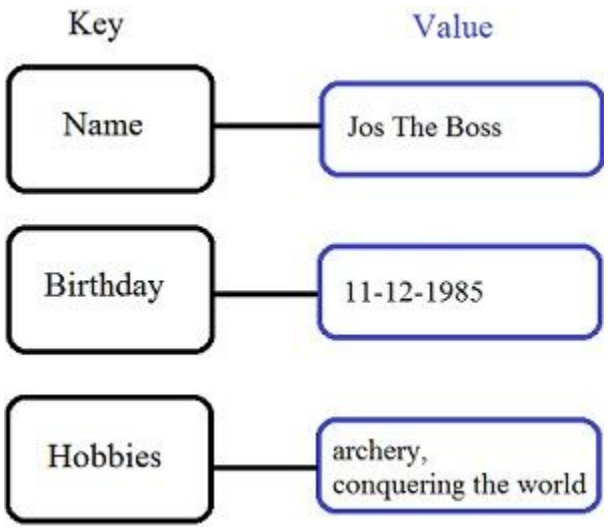
Key-value Stores

- Basic data model with a collection of key-value pairs.
- Data is stored as unstructured values or blobs associated with a unique key.
- Simple and fast retrieval of values based on keys.
- Examples: Redis, Amazon DynamoDB, Riak

Key	Value
K1	AAA,BBB,CCC
K2	AAA,BBB
K3	AAA,DDD
K4	AAA,2,01/01/2015
K5	3,ZZZ,5623

Key/Value stores

- Store data in a schema-less way
- Store data as maps
  - HashMaps or associative arrays
  - Provide a very efficient average running time algorithm for accessing data
- Notable for:
  - Couchbase (Zynga, Vimeo, NAVTEQ, ...)
  - Redis (Craiglist, Instagram, StackOverfow, flickr, ...)
  - Amazon Dynamo (Amazon, Elsevier, IMDb, ...)
  - Apache Cassandra (Facebook, Digg, Reddit, Twitter,...)
  - Voldemort (LinkedIn, eBay, …)
  - Riak (Github, Comcast, Mochi, ...)



Key Value Stores

- Key-Value Stores are a type of **data store** that organise data differently from your traditional SQL store.
- The fundamental data model of a key-value store is **the associative array** (a.k.a. a map, a dictionary or a hash). It is a **collection of key-value pairs**, where the **key is unique** in the collection.
- A key can be an **ID or a name** or anything you want to use as an identifier.
- Rather than storing data into a variety of tables and columns like in SQL stores, **key-value stores split a data model into a collection of data structures such as, key-value strings, lists, hashes, sets,** etc.
- **Redis** focuses on **high performance and a simple querying language** that is just a set of data retrieval commands.

Key Value Stores

- The nature of key-value stores makes them **best suited to operate as caches or data structure stores and in situations that are performance sensitive**.
- We can build more **advanced data structures on top of key-value pairs**. You can also use the **high performance to build queues or publish-subscribe mechanisms.**
- Key-value stores **fall into the NoSQL family of databases**, they do not use SQL and have a **flexible schema**.
- Application defines the **key-value pairs** and can **change the definition** at any time. We decide **how to store** your data.

Key Value Stores

A quick overview of key-value stores

- **Key-value stores** are one of the **simplest forms of database**. Almost all programming languages come with **in-memory key-value stores**. The map container from the **C++ STL** is a key-value store, just like the **HashMap of Java**, and the **dictionary type in Python**. Key-value stores generally share the following interface:

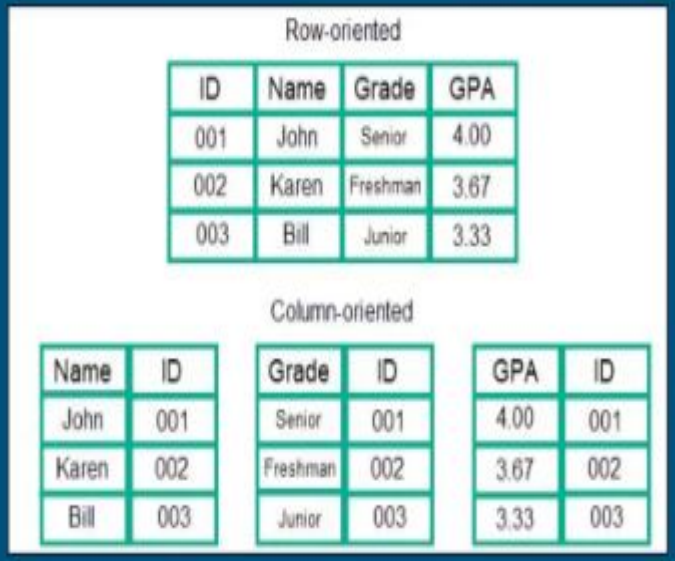
- **Get(key):** Get some data previously saved under the identifier "key", or fail if no data was stored for "key".
- **Set(key, value):** Store the "value" in memory under the identifier "key", so we can access it later referencing the same "key". If some data was already present under the "key", this data will be replaced.

- **Delete(key):** Delete the data that was stored under the "key".

Column-Oriented

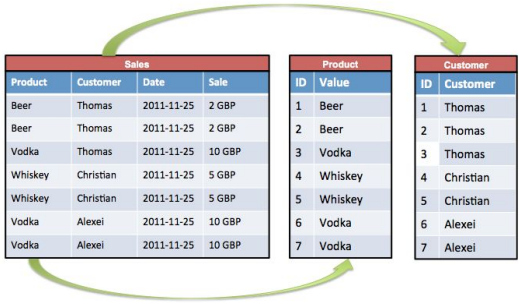
Column-oriented Database

- Data is organized into columns instead of rows.
- Columns are grouped into column families, and each column family can have a different set of columns.
- Well-suited for read and write intensive workloads.
- Example: Apache Cassandra, HBase , Amazon SimpleDB



Column-oriented storage

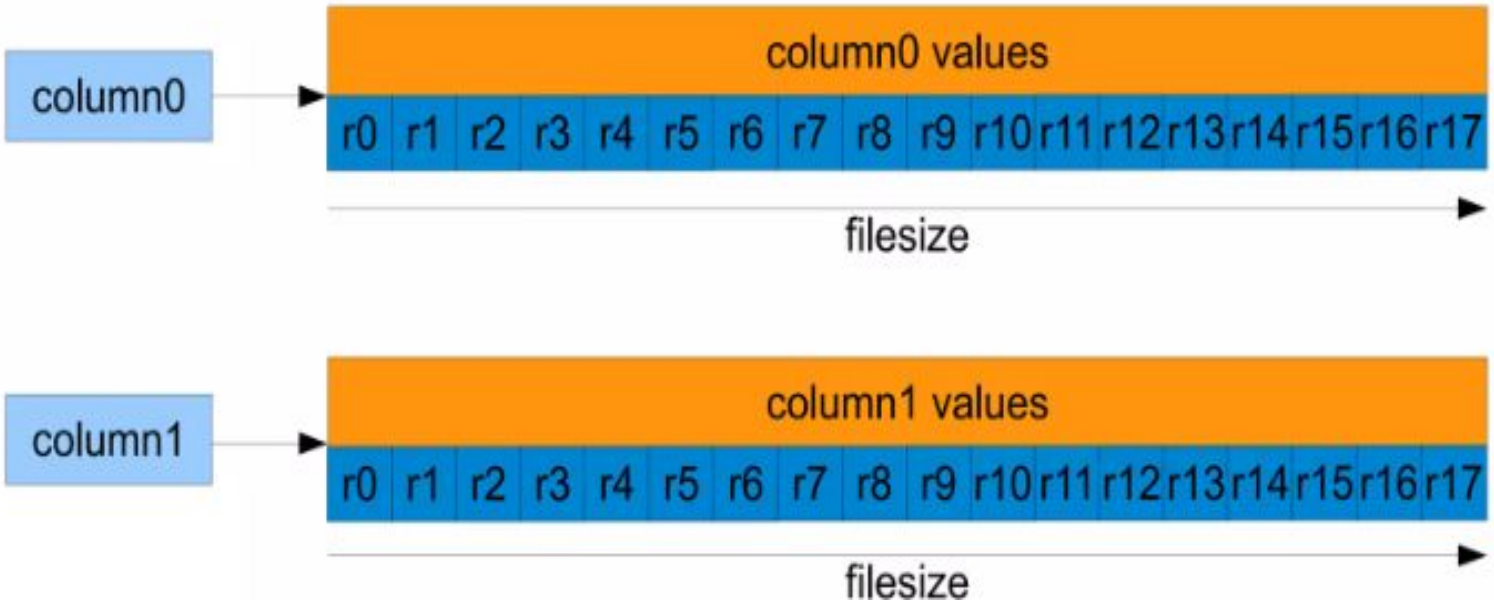
- Column stores store data in column-specific files
- Simplest case: one datafile per column
- Row values for each column are stored contiguously

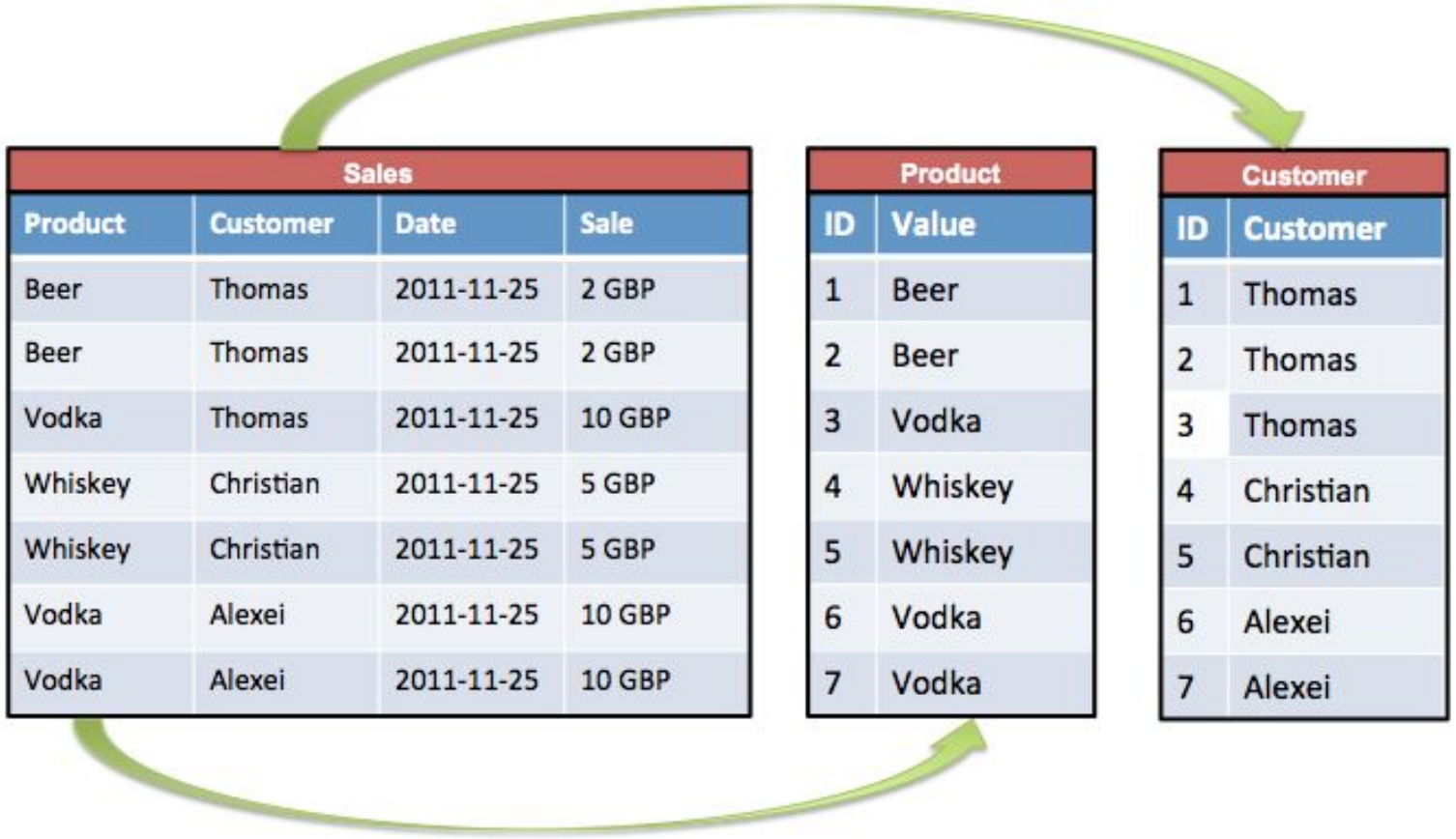


Sales			
Product	Customer	Date	Sale
Beer	Thomas	2011-11-25	2 GBP
Beer	Thomas	2011-11-25	2 GBP
Vodka	Thomas	2011-11-25	10 GBP
Whiskey	Christian	2011-11-25	5 GBP
Whiskey	Christian	2011-11-25	5 GBP
Vodka	Alexei	2011-11-25	10 GBP
Vodka	Alexei	2011-11-25	10 GBP

Product	
ID	Value
1	Beer
2	Beer
3	Vodka
4	Whiskey
5	Whiskey
6	Vodka
7	Vodka

Customer	
ID	Customer
1	Thomas
2	Thomas
3	Thomas
4	Christian
5	Christian
6	Alexei
7	Alexei





Sales

Product	Customer	Date	Sale
Beer	Thomas	2011-11-25	2 GBP
Beer	Thomas	2011-11-25	2 GBP
Vodka	Thomas	2011-11-25	10 GBP
Whiskey	Christian	2011-11-25	5 GBP
Whiskey	Christian	2011-11-25	5 GBP
Vodka	Alexei	2011-11-25	10 GBP
Vodka	Alexei	2011-11-25	10 GBP

Product

ID	Value
1	Beer
2	Beer
3	Vodka
4	Whiskey
5	Whiskey
6	Vodka
7	Vodka

Customer

ID	Customer
1	Thomas
2	Thomas
3	Thomas
4	Christian
5	Christian
6	Alexei
7	Alexei

Column-oriented storage

- Column stores can greatly improve the performance of queries that only touch a small amount of columns
- This is because they will only access these columns' particular data
- Simple math: table t has a total of 10 GB data, with
  - column a: 4 GB
  - column b: 2 GB
  - column c: 3 GB
  - column d: 1 GB
- If a query only uses column d, at most 1 GB of data will be processed by a column store
- In a row store, the full 10 GB will be processed

Row Store and Column Store

- Most of the queries does not process all the attributes of a particular relation.

- For example the query

```
Select c.name and c.address
From CUSTOMES as c
Where c.region=Mumbai;
```

- Only process three attributes of the relation CUSTOMER. But the customer relation can have more than three attributes.

- Column-stores are more I/O efficient for read-only queries as they read, only those attributes which are accessed by a query.

Row Store and Column Store

Row Store	Column Store
(+) Easy to add/modify a record	(+) Only need to read in relevant data
(-) Might read in unnecessary data	(-) Tuple writes require multiple accesses

- So column stores are suitable for read-mostly, read-intensive, large data repositories

Why Column Stores?

- Can be significantly faster than row stores for some applications
  - Fetch only required columns for a query
  - Better cache effects
  - Better compression (similar attribute values within a column)
- But can be slower for other applications
  - OLTP with many row inserts, ..
- Long war between the column store and row store camps :-)
  - This paper tries to give a balanced picture of advantages and disadvantages, after adding/ subtracting a number of optimizations for each approach

Column Stores - Data Model

- Standard relational logical data model
  - EMP(name, age, salary, dept)
  - DEPT(dname, floor)
- Table – collection of projections
- Projection – set of columns
- Horizontally partitioned into segments with segment identifier

Column Stores - Data Model

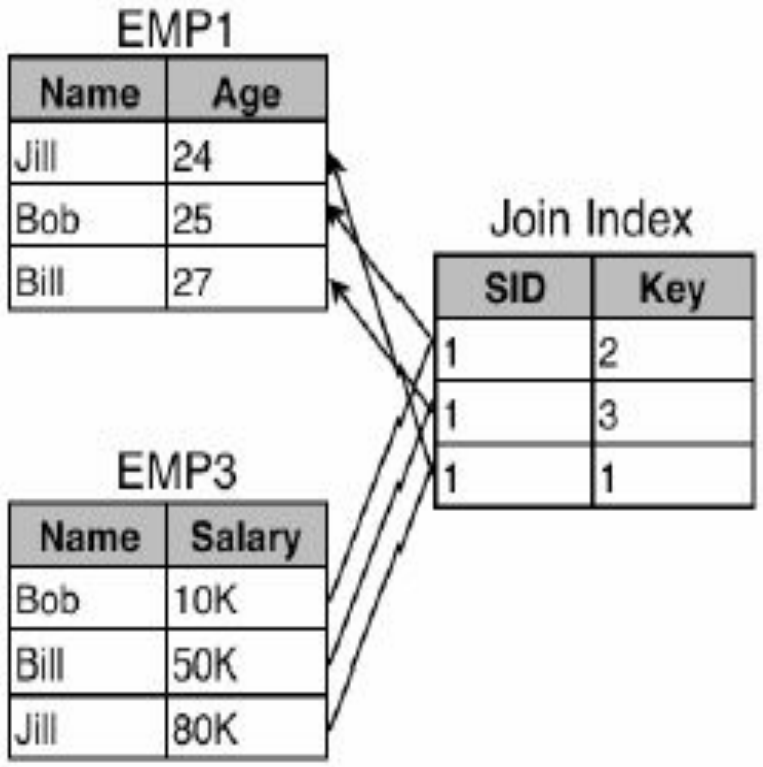
- To answer queries, projections are joined using Storage keys and join indexes
- Storage Keys:
  - Within a segment, every data value of every column is associated with a unique Skey
  - Values from different columns with matching Skey belong to the same logical row

Column Stores – Data Model

- Join Indexes
  - T1 and T2 are projections on T
  - M segments in T1 and N segments in T2
  - Join Index from T1 to T2 is a table of the form:
    - (s: Segment ID in T2, k: Storage key in Segment s)
    - Each row in join index matches corresponding row in T1
  - Join indexes are built such that T could be efficiently reconstructed from T1 and T2

Column Stores – Data Model

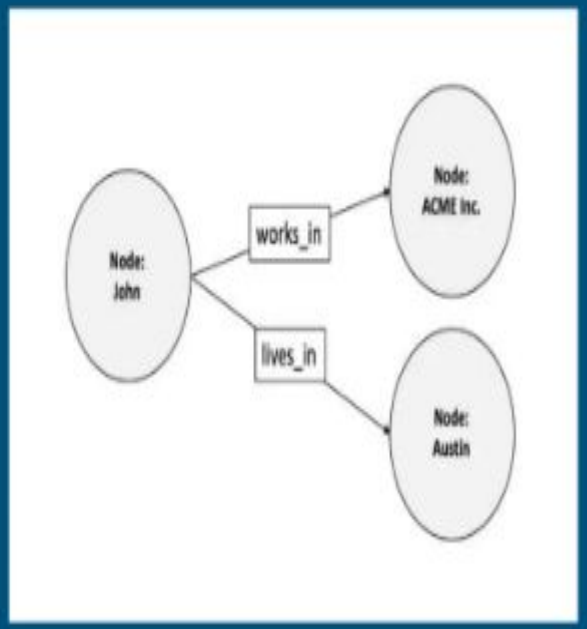
- Construct EMP(name, age, salary) from EMP1 and EMP3 using join index on EMP3



Graph-Based

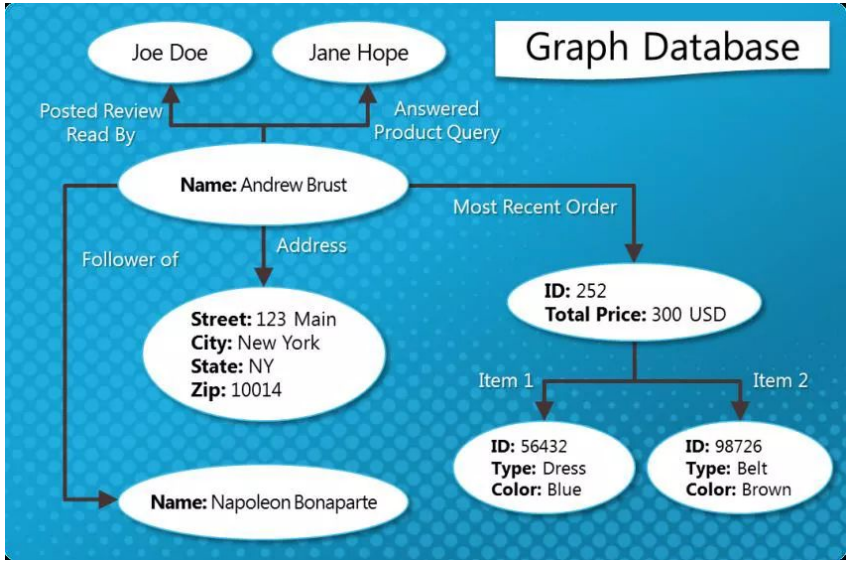
Graph-based database

- Designed for handling relationships between data points.
- Data is represented as nodes, edges, and properties.
- Efficient for traversing and querying complex networks or graphs.
- Examples: Neo4j, ArangoDB, Amazon Neptune



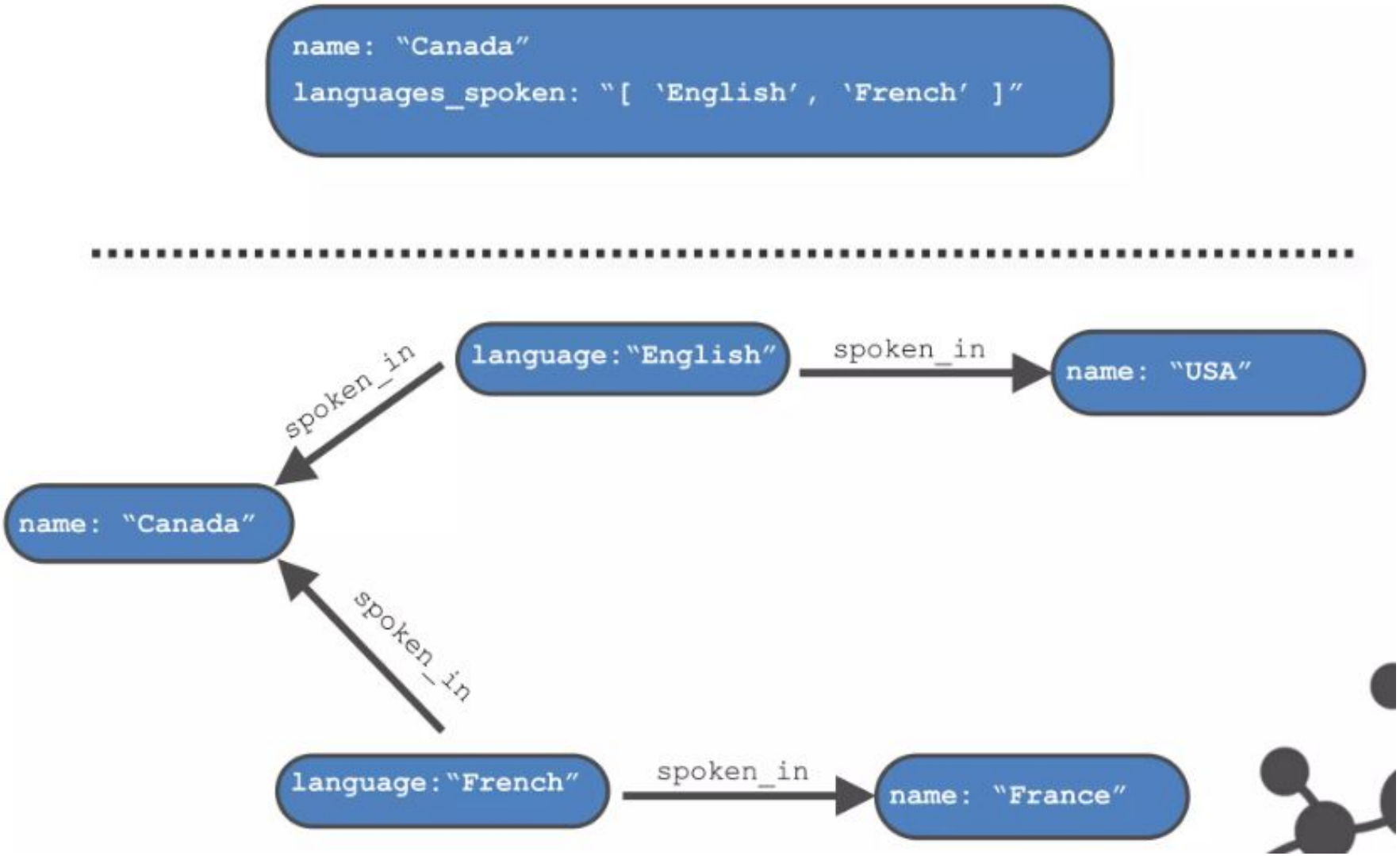
Graph Databases

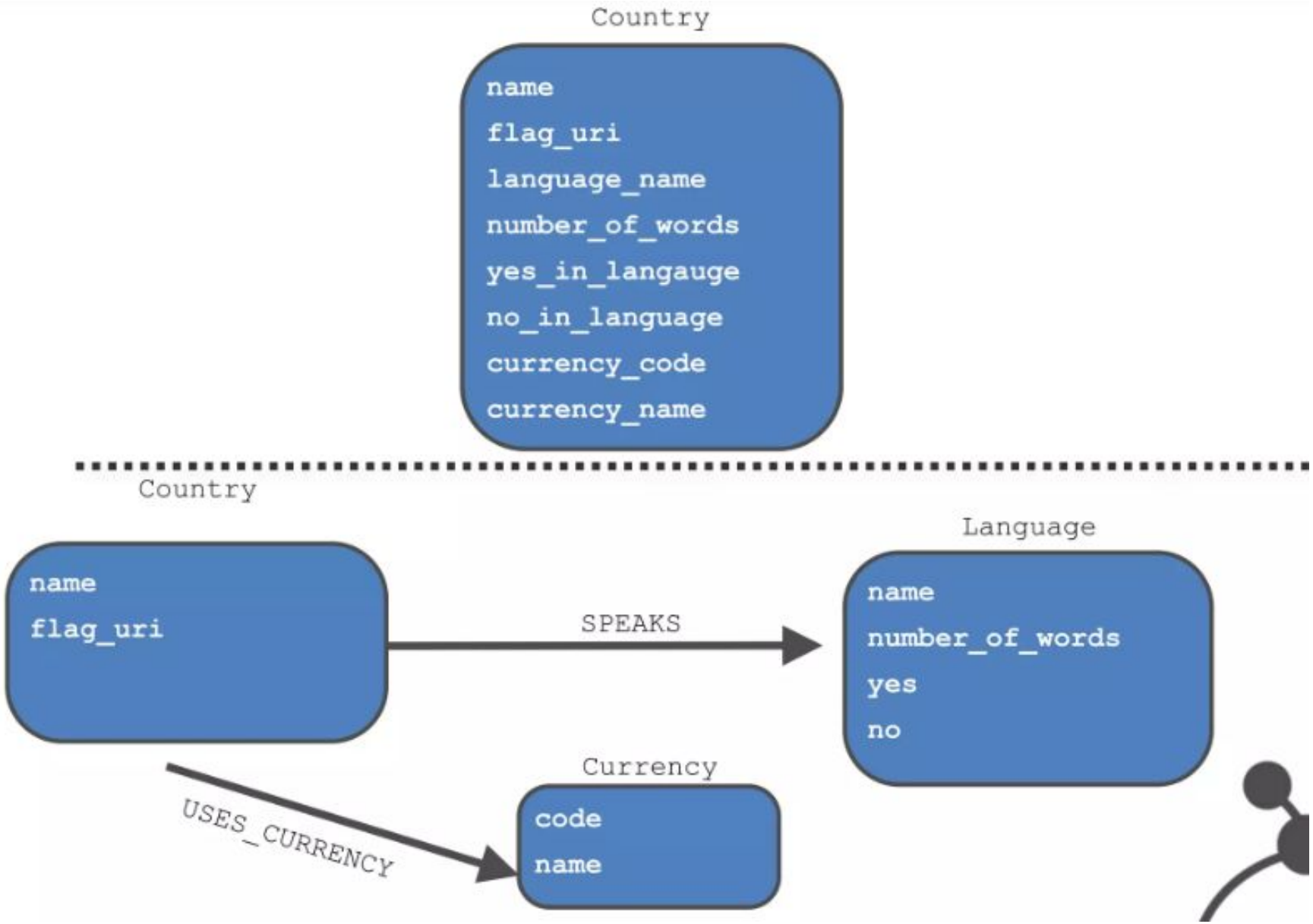
- Graph-oriented
- Everything is stored as an edge, a node or an attribute.
- Each node and edge can have any number of attributes.
- Both the nodes and edges can be labelled.
- Labels can be used to narrow searches.



What is Neo4j?

- A Graph Database + Lucene Index
- Property Graph
- Full ACID
  (atomicity, consistency, isolation, durability)
- High Availability (with Enterprise Edition)
- 32 Billion Nodes, 32 Billion Relationships,
  64 Billion Properties
- Embedded Server
- REST API


name: "Canada"
languages_spoken: "['English', 'French']"
language:"English"
spoken_in
name: "USA"
spoken_in
name: "Canada"
spoken_in
language:"French"
spoken_in
name: "France"


Country
name
flag_uri
language_name
number_of_words
yes_in_langauge
no_in_language
currency_code
currency_name
Country
name
flag_uri
SPEAKS
Language
name
number_of_words
yes
no
USES_CURRENCY
Currency
code
name

Pattern Matching Query Language (like SQL for graphs)

```
// get node 0

start a=(0) return a

// traverse from node 1

start a=(1) match (a)-->(b) return b

// return friends of friends

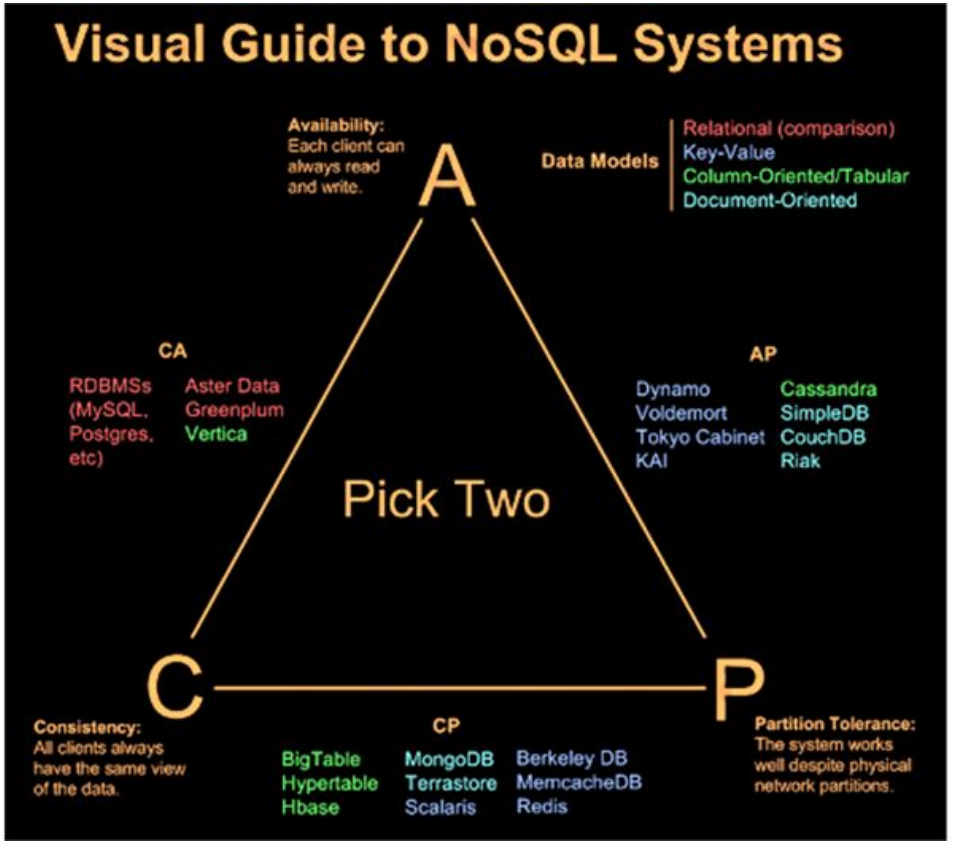
start a=(1) match (a)--()--(c) return c
```

CAP Theorem

A congruent and logical way for assessing the problems involved in assuring ACID-like guarantees in distributed systems is provided by the CAP theorem

At most two of the following three can be maximized at one time

- Consistency
  - Each client has the same view of the data
- Availability
  - Each client can always read and write
- Partition tolerance
  - System works well across distributed physical networks



CAP Theorem: Two out of Three

- CAP theorem – At most two properties on three can be addressed
- The choices could be as follows:

1. Availability is compromised but consistency and partition tolerance are preferred over it
2. The system has little or no partition tolerance. Consistency and availability are preferred
3. Consistency is compromised but systems are always available and can work when parts of it are partitioned

Consistency or Availability

- Consistency and Availability is not "binary" decision
- AP systems relax consistency in favor of availability – but are not inconsistent
- CP systems sacrifice availability for consistency- but are not unavailable
- This suggests both AP and CP systems can offer a degree of consistency, and availability, as well as partition tolerance

